Aula 4 - Orquestração de Containers e Ambientes com Helm (Kubernetes)

Uma jornada completa pelo gerenciamento profissional de aplicações em ambientes distribuídos

Made with GAMMA

O que é Kubernetes e por que orquestrar containers?

Definição e Conceitos Fundamentais

Kubernetes, frequentemente abreviado como K8s, é uma plataforma open source poderosa desenvolvida pelo Google para automatizar a implantação, o dimensionamento (scaling) e a gestão de aplicações em containers. Essa tecnologia revolucionou a forma como empresas gerenciam suas cargas de trabalho em nuvem e ambientes distribuídos.

A orquestração de containers é o processo de coordenação automática de múltiplos containers, garantindo que eles funcionem de forma harmoniosa e eficiente. O Kubernetes atua como um "maestro digital", gerenciando toda a orquestra de containers para fornecer alta disponibilidade, escalabilidade e resiliência.

Auto-recuperação

Reinício automático de containers que falham, garantindo continuidade do serviço

Balanceamento de carga

Distribuição inteligente do tráfego entre múltiplos pods para otimização de recursos

Escalabilidade

Expansão e redução automática de recursos conforme demanda do sistema

Atualizações contínuas

Deploy seguro com rollback automático em caso de problemas

Esses benefícios transformam a gestão de infraestrutura em uma operação ágil, confiável e escalável, essencial para aplicações modernas que precisam atender milhões de usuários simultaneamente.

Made with GAMMA

Helm: O gerenciador de pacotes para Kubernetes

01

Simplificação do Deploy

Helm abstrai a complexidade de gerenciar múltiplos arquivos YAML do Kubernetes, consolidando tudo em uma única interface intuitiva

02

Uso de Charts

Charts são pacotes que definem recursos e configurações do Kubernetes, incluindo templates, valores padrão e dependências

03

Versionamento Inteligente

Mantenha histórico completo de suas implantações, permitindo rollback instantâneo para qualquer versão anterior

04

Customização Flexível

Adapte ambientes de desenvolvimento, teste e produção através de valores personalizados sem modificar os templates

Vantagens do Helm

- Redução drástica de erros humanos
- Reutilização de templates entre projetos
- Gestão centralizada de configurações
- Compartilhamento fácil de charts via repositórios

Casos de Uso Comuns

- Deploy de aplicações complexas com múltiplos serviços
- Gestão de ambientes diferenciados
- Automatização de pipelines CI/CD
- Padronização entre times e projetos

Made with GAMMA

Passo a passo básico com Helm

A instalação e configuração inicial do Helm é um processo simples, mas fundamental para começar a trabalhar com orquestração de containers no Kubernetes.

Instalação do Helm CLI

Execute o script oficial do Helm no terminal: `curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3` e depois `chmod 700 get_helm.sh && ./get_helm.sh`

Configuração do Cluster

Verifique se o kubectl está configurado corretamente com seu cluster Kubernetes usando `kubectl cluster-info`. Este comando mostra informações sobre o cluster atual

Primeiro Deploy

Utilize o comando `helm install minha-aplicacao ./chart` para implantar sua aplicação. Substitua "minha-aplicacao" pelo nome desejado e "./chart" pelo caminho do seu chart

Atualização de Versão

Para atualizar uma implantação existente, use `helm upgrade minha-aplicacao ./chart`. Este comando aplica as mudanças definidas no chart

Rollback de Emergência

Em caso de problemas, reverta para uma versão anterior com `helm rollback minha-aplicacao [revision]`, onde [revision] é o número da revisão desejada

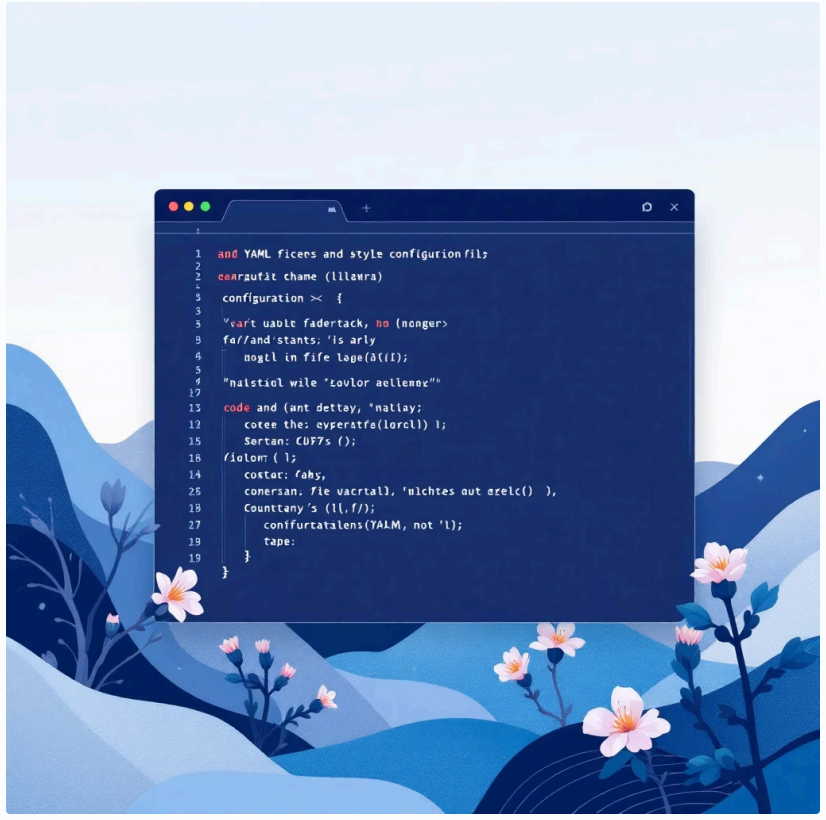
Dica Profissional: Sempre liste as releases com `helm list` antes de fazer rollback para verificar qual revisão está ativa. Use `helm history minha-aplicacao` para ver o histórico completo de mudanças.

Made with GAMMA

Definindo ambientes com Helm: values.yaml e customizações

O arquivo `values.yaml` é o coração da configuração do Helm. Ele permite parametrizar sua aplicação para diferentes ambientes sem modificar os templates principais. Esta separação entre código e configuração é fundamental para boas práticas DevOps.

Este arquivo usa formato YAML para definir variáveis que podem ser referenciadas nos templates. Cada ambiente (desenvolvimento, teste, produção) pode ter seu próprio arquivo de valores, mantendo a mesma estrutura de templates mas com configurações específicas.



Estrutura Básica do values.yaml

O exemplo mostrado configura uma aplicação com 2 réplicas, usando uma imagem Docker específica e expondo o serviço na porta 80 através de um LoadBalancer. Esta configuração pode ser facilmente ajustada para cada ambiente.

```yaml
replicaCount: 2

image:
  repository: minha-imagem
  tag: "latest"
  pullPolicy: IfNotPresent

service:
  type: LoadBalancer
  port: 80
  targetPort: 8080

resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 250m
    memory: 256Mi

env:
  - name: DATABASE_URL
    value: "postgres://db:5432"
  - name: LOG_LEVEL
    value: "info"
```

Valores por Ambiente

Crie arquivos como `values-dev.yaml`, `values-staging.yaml` e `values-prod.yaml` com configurações específicas para cada ambiente

Uso na Implantação

Execute `helm install minha-aplicacao ./chart -f values-prod.yaml` para usar configurações específicas de produção

Sobrescrita de Valores

Use `--set` para sobrescrever valores individualmente: `helm install minha-aplicacao ./chart --set replicaCount=3`

Made with GAMMA

Desafio Técnico 3: Objetivos e metodologia

Trabalho em Equipe

Desenvolver colaborativamente o deploy orquestrado usando Helm no Kubernetes, promovendo troca de conhecimentos e resolução conjunta de problemas

Charts Personalizados

Criar charts reutilizáveis para aplicação com múltiplos ambientes, seguindo as melhores práticas de estruturação e organização

Apresentação de Resultados

Desenvolver slides profissionais demonstrando a solução implementada, incluindo arquitetura, comandos utilizados e resultados alcançados

Relatório Técnico

Entregar documentação completa no repositório oficial da disciplina, incluindo código, configurações e análise de aprendizados

Metodologia de Desenvolvimento

Fase 1: Planejamento

1. Definir arquitetura da aplicação
2. Identificar recursos do Kubernetes necessários
3. Planejar estrutura de diretórios do chart
4. Distribuir tarefas entre membros da equipe

Fase 2: Implementação

1. Criar templates para cada recurso
2. Desenvolver arquivos values.yaml por ambiente
3. Testar deploy localmente
4. Ajustar configurações conforme necessário

Fase 3: Validação

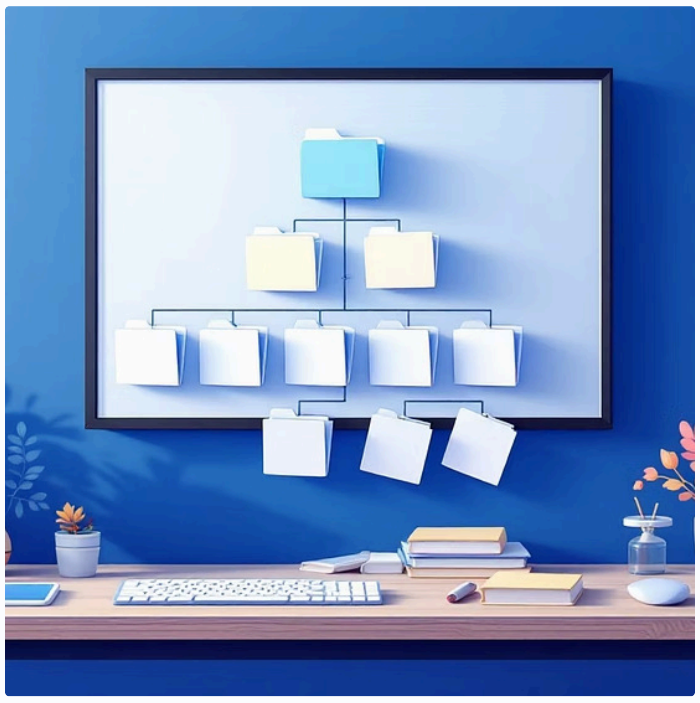
1. Testar funcionalidade completa
2. Realizar testes de upgrade e rollback
3. Documentar processos
4. Preparar apresentação

Repositório Oficial: Acesse e entregue suas contribuições em https://github.com/profclovesrocha/devops/tree/main/desafios. Siga as convenções de nomenclatura e estrutura definidas na disciplina.

Made with GAMMA

Demonstração prática do desafio

Esta seção apresenta uma visão prática da implementação do Desafio 3, mostrando a estrutura de diretórios, comandos utilizados e resultados obtidos durante o desenvolvimento.

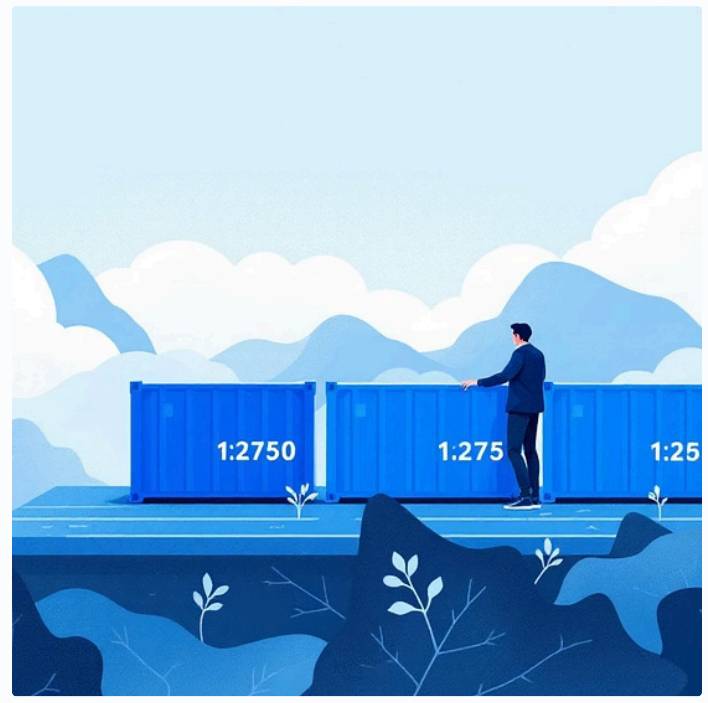


Estrutura de Diretórios

Organização do chart com templates/, charts/, values.yaml e Chart.yaml seguindo padrões Helm

Comandos de Deploy

Execução de helm install, helm upgrade e helm rollback com outputs de sucesso



Dashboard Kubernetes

Visualização dos pods, serviços e deployments ativos no cluster através do dashboard

Exemplo de Chart Criado pela Equipe

Estrutura de Arquivos

```
meu-chart/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl
└── charts/
    └── (dependências)
```

Cada arquivo tem responsabilidade específica: Chart.yaml define metadados, templates/ contém os manifests do Kubernetes parametrizados, values.yaml configura valores padrão, e charts/ armazena dependências.

Comandos Executados

```
# Instalação inicial
helm install meu-app ./meu-chart

# Upgrade com novas configurações
helm upgrade meu-app ./meu-chart \
  --set replicaCount=3

# Verificar status
helm status meu-app

# Rollback para revisão anterior
helm rollback meu-app 1
```

Estes comandos foram testados em diferentes ambientes para garantir funcionamento correto.

Resultados no Dashboard Kubernetes

Durante a execução do desafio, o dashboard do Kubernetes foi utilizado para monitorar a saúde da aplicação. Os pods foram criados conforme definido no deployment, com status Running após alguns segundos. O serviço LoadBalancer foi provisionado corretamente, expondo a aplicação externamente. O número de réplicas correspondeu aos valores configurados em cada ambiente.

Made with GAMMA

Principais aprendizados e dificuldades enfrentadas

O Desafio 3 proporcionou uma experiência prática valiosa em orquestração de containers com Helm e Kubernetes. A equipe enfrentou desafios típicos de projetos reais, mas conseguiu superá-los através de colaboração e pesquisa.

Importância da Parametrização

Aprendemos que separar configuração de código através de values.yaml é fundamental. Isso permite reutilizar os mesmos templates para múltiplos ambientes, apenas alterando valores específicos. A parametrização também facilita testes e ajustes sem modificar a estrutura dos manifests.

Desafios com LoadBalancer

A configuração de serviços LoadBalancer apresentou dificuldades iniciais, especialmente em clusters locais como Minikube. Descobrimos que LoadBalancer requer infraestrutura cloud ou alternativas como MetalLB. Optamos por usar NodePort em ambientes locais e LoadBalancer apenas em produção.

Configuração de Ingressos

Os ingressos, que controlam o acesso externo aos serviços, exigiram configuração cuidadosa de regras e anotações. Precisamos instalar um controller de ingress (nginx-ingress) e entender como definir hosts, paths e TLS. Este foi um dos pontos mais complexos do desafio.

Benefícios da Automação

Reconhecemos os benefícios do Helm para automação e padronização. O versionamento de releases e rollback rápido transformaram a gestão de deploys. A capacidade de revisar mudanças através do histórico e reverter instantaneamente em falhas aumentou confiança no processo.

Lições Aprendidas Valiosas

- Documentação clara é essencial: comentar templates e explicar valores facilita manutenção
- Testar em ambientes menores antes de produção: Minikube é perfeito para validação inicial
- Usar Helm hooks para operações especiais: pré-install, post-upgrade permitem lógica customizada
- Validar templates antes de instalar: helm template e helm lint evitam erros de sintaxe
- Gerenciar segredos adequadamente: nunca hardcode senhas em values.yaml, usar Secrets do Kubernetes

Made with GAMMA

Recomendações para o uso avançado de Helm e Kubernetes

Com os fundamentos consolidados, é hora de explorar recursos avançados que elevam a maturidade de sua implementação Kubernetes e otimizam operações de produção.

Templates Helm Avançados

Explore funcionalidades como templates condicionais com if/else, loops com range, variáveis e pipeline. Use `_helpers.tpl` para criar blocos reutilizáveis. Implemente subcharts para composição modular de aplicações complexas. Aproveite funções do Sprig para manipulação de strings e dados.

Integração com CI/CD

Automatize deploys através de pipelines GitLab CI, GitHub Actions ou Jenkins. Configure validação automática de templates antes de instalar. Implemente estratégias de canary deployment ou blue-green para releases seguras. Integre testes de integração para verificar funcionalidade pós-deploy.

Monitoramento e Alertas

Instale Prometheus e Grafana para métricas de cluster e aplicações. Configure Alertmanager para notificações em problemas. Use kubectl top para análise de recursos. Implemente readiness e liveness probes para auto-recuperação. Monitore custos de recursos para otimização financeira.

Ferramentas e Extensões Essenciais

Segurança com OPA

Open Policy Agent para validação de políticas de segurança antes de deploys

Gerenciamento de Segredos

External Secrets para integração com vaults externos como Hashicorp Vault ou AWS Secrets Manager

GitOps com Argo CD

Sincronização automática de estado declarativo no Git com cluster em tempo real

Boas Práticas de Produção

Organização de Código

- Versione seus charts semantically
- Mantenha um repositório Helm central
- Documente valores e templates
- Use Chart.yaml para dependências

Gestão de Ambientes

- Use namespaces para isolamento
- Implemente approval workflows
- Configure quotas de recursos
- Aplique labels para organização

Made with GAMMA

Conclusão e próximos passos

Consolidação de Conhecimentos

Helm e Kubernetes são ferramentas essenciais no toolkit DevOps moderno, proporcionando orquestração profissional de containers

Trabalho Colaborativo

O Desafio 3 consolidou habilidades práticas e trabalho em equipe, preparando para cenários profissionais reais

Continuidade de Aprendizado

Incentivamos exploração contínua através de contribuições no repositório oficial e estudos avançados

Próximos Passos Recomendados

Estudos Avançados

Aprofunde em CRDs (Custom Resource Definitions) e Operators para extensões personalizadas do Kubernetes

Service Mesh

Explore Istio ou Linkerd para gerenciamento avançado de microserviços e comunicação entre serviços

Serverless

Investigue Knative para execução serverless em Kubernetes, combinando benefícios de containers com escalabilidade

Contribuição no Repositório Oficial

Convidamos todos os estudantes a contribuir ativamente no repositório da disciplina: **https://github.com/profclovesrocha/devops/tree/main/desafios**. Suas contribuições podem incluir:

- Documentação de novos charts desenvolvidos
- Exemplos de implementação de padrões
- Tutoriais passo a passo sobre tópicos específicos
- Scripts de automação para deploys
- Análises de caso de uso reais

Perguntas e Discussão

Estamos à disposição para esclarecer dúvidas sobre qualquer aspecto do conteúdo apresentado. Seja sobre conceitos teóricos, comandos específicos, problemas enfrentados durante o desafio ou direcionamentos para aprofundamento, aproveite este momento para trocar experiências com a turma e esclarecer pontos que ainda geram dúvidas. A colaboração e discussão são fundamentais para o aprendizado contínuo!

Made with GAMMA